Word2vec learns dense embeddings from context

Running text supplies the labels; the learned classifier weights become word vectors.

Sparse count vector

`[0, 0, 442, 0, 8, ...]`

$|V|$ dimensions · mostly zeroseach dimension names a context word

learn rather than count

Dense word2vec embedding

`[0.31, -0.08, 0.74, ...]`

$d \approx 50\text{–}1000$ · dense real valueslearned dimensions need not be interpretable

Word2vec learns dense embeddings from context

Running text supplies the labels; the learned classifier weights become word vectors.

Sparse count vector

`[0, 0, 442, 0, 8, ...]`

$|V|$ dimensions · mostly zeroseach dimension names a context word

learn rather than count →

Dense word2vec embedding

`[0.31, -0.08, 0.74, ...]`

$d \approx 50\text{–}1000$ · dense real valueslearned dimensions need not be interpretable

Compact vectors reduce parameters and may capture synonymy better.

Word2vec learns dense embeddings from context

Running text supplies the labels; the learned classifier weights become word vectors.

Sparse count vector

`[0, 0, 442, 0, 8, ...]`

$|V|$ dimensions · mostly zeroseach dimension names a context word

learn rather than count

Dense word2vec embedding

`[0.31, -0.08, 0.74, ...]`

$d \approx 50\text{–}1000$ · dense real valueslearned dimensions need not be interpretable

Compact vectors reduce parameters and may capture synonymy better.

Skip-gram with negative sampling (SGNS)

Running text creates the labels: self-supervision

Word2vec learns dense embeddings from context

Running text supplies the labels; the learned classifier weights become word vectors.

Sparse count vector

`[0, 0, 442, 0, 8, ...]`

$|V|$ dimensions · mostly zeroseach dimension names a context word

learn rather than count

Dense word2vec embedding

`[0.31, -0.08, 0.74, ...]`

$d \approx 50\text{–}1000$ · dense real valueslearned dimensions need not be interpretable

Compact vectors reduce parameters and may capture synonymy better.

Skip-gram with negative sampling (SGNS)

Running text creates the labels: self-supervision

1 Positive pairs

Nearby words
(apricot, jam) → +

Word2vec learns dense embeddings from context

Running text supplies the labels; the learned classifier weights become word vectors.

Sparse count vector

`[0, 0, 442, 0, 8, ...]`

$|V|$ dimensions · mostly zeroseach dimension names a context word

learn rather than count →

Dense word2vec embedding

`[0.31, -0.08, 0.74, ...]`

$d \approx 50\text{–}1000$ · dense real valueslearned dimensions need not be interpretable

Compact vectors reduce parameters and may capture synonymy better.

Skip-gram with negative sampling (SGNS)

Running text creates the labels: self-supervision

1 Positive pairs

Nearby words (apricot, jam) → +

→

2 Negative pairs

Random samples (apricot, aardvark) → -

→

Word2vec learns dense embeddings from context

Running text supplies the labels; the learned classifier weights become word vectors.

Sparse count vector

`[0, 0, 442, 0, 8, ...]`

$|V|$ dimensions · mostly zeroseach dimension names a context word

learn rather than count →

Dense word2vec embedding

`[0.31, -0.08, 0.74, ...]`

$d \approx 50\text{–}1000$ · dense real valueslearned dimensions need not be interpretable

Compact vectors reduce parameters and may capture synonymy better.

Skip-gram with negative sampling (SGNS)

Running text creates the labels: self-supervision

1 Positive pairs

Nearby words (apricot, jam) → +

→

2 Negative pairs

Random samples (apricot, aardvark) → -

→

3 Binary classifier

Logistic regression separates + from -

→

Word2vec learns dense embeddings from context

Running text supplies the labels; the learned classifier weights become word vectors.

Sparse count vector

`[0, 0, 442, 0, 8, ...]`

$|V|$ dimensions · mostly zeroseach dimension names a context word

learn rather than count →

Dense word2vec embedding

`[0.31, -0.08, 0.74, ...]`

$d \approx 50\text{–}1000$ · dense real valueslearned dimensions need not be interpretable

Compact vectors reduce parameters and may capture synonymy better.

Skip-gram with negative sampling (SGNS)

Running text creates the labels: self-supervision

1 Positive pairs

Nearby words (apricot, jam) → +

→

2 Negative pairs

Random samples (apricot, aardvark) → -

→

3 Binary classifier

Logistic regression separates + from -

→

4 Keep the weights

One fixed vector per word (static embedding)

Skip-gram asks whether two words belong together

A target word and a candidate context word define one binary classification example.

A ±2 context window supplies observed pairs

…	lemon,	a	tablespoon	of	apricot	jam,	a	pinch	…
			c_1	c_2	target w	c_3	c_4		

Skip-gram asks whether two words belong together

A target word and a candidate context word define one binary classification example.

A ±2 context window supplies observed pairs

…	lemon,	a	tablespoon	of	apricot	jam,	a	pinch	…
			c_1	c_2	target w	c_3	c_4		

Observed nearby

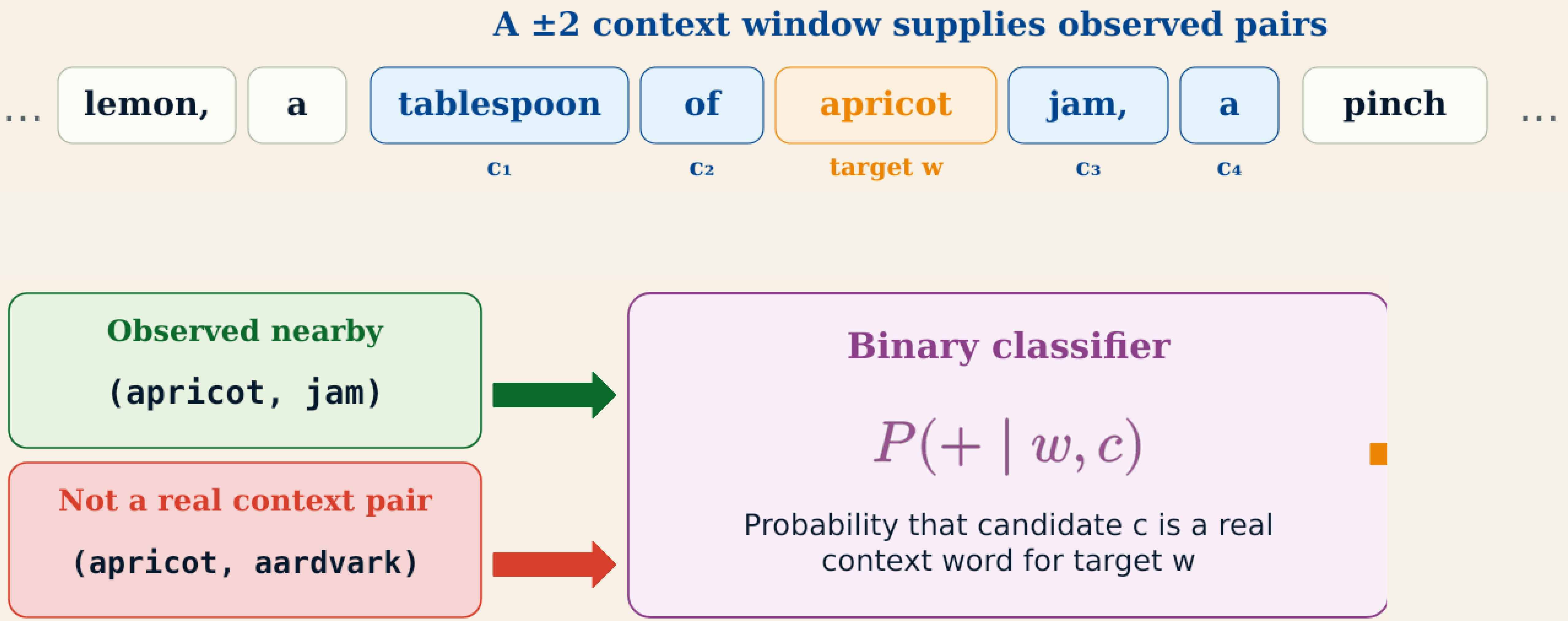
`(apricot, jam)`

Not a real context pair

`(apricot, aardvark)`

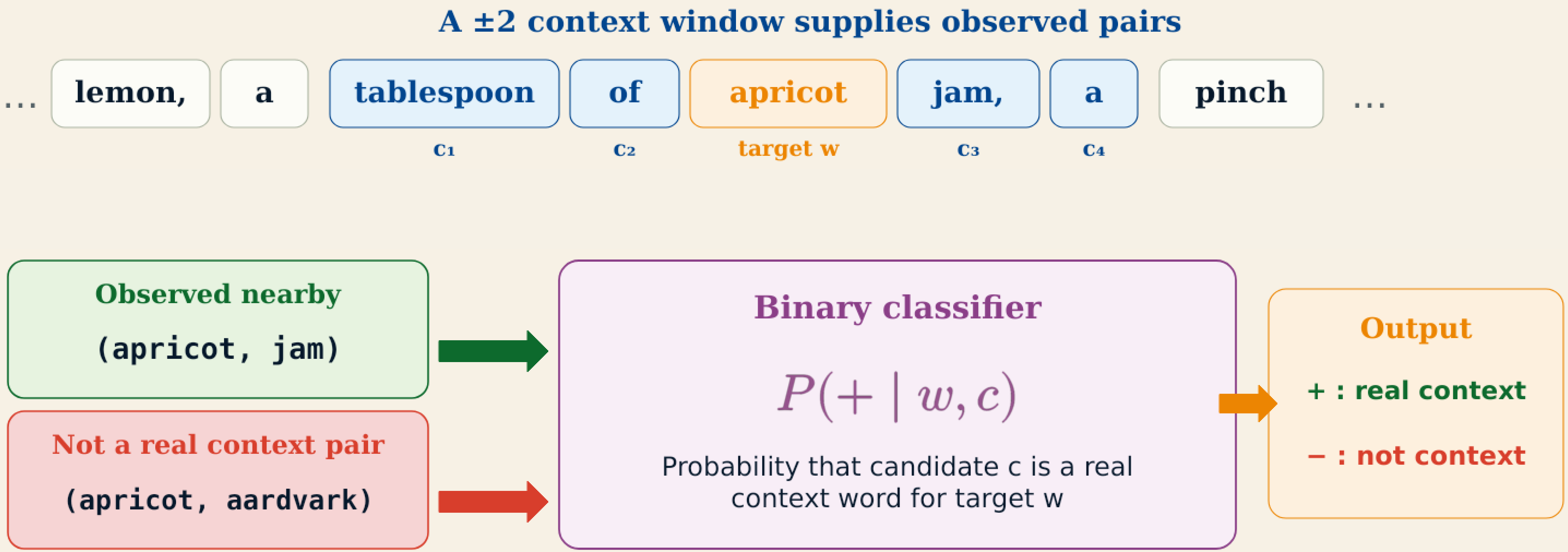
Skip-gram asks whether two words belong together

A target word and a candidate context word define one binary classification example.



Skip-gram asks whether two words belong together

A target word and a candidate context word define one binary classification example.



Skip-gram asks whether two words belong together

A target word and a candidate context word define one binary classification example.

A ±2 context window supplies observed pairs

… lemon, a **tablespoon** **of** **apricot** **jam,** **a** pinch …

c_1 (tablespoon) · c_2 (of) · target w (apricot) · c_3 (jam,) · c_4 (a)

Observed nearby

`(apricot, jam)`

Not a real context pair

`(apricot, aardvark)`

Binary classifier

$$P(+ \mid w, c)$$

Probability that candidate c is a real context word for target w

Output

+ : real context

− : not context

J&M §5.5.1

Where do the vectors *w* and *c* used by this classifier come from?

Every word has a target and a context embedding

Skip-gram learns separate parameter matrices W and C for these two roles.

$\theta =$

W target	dimensions 1 … d
aardvark	· · … ·
apricot	0.31 −0.08 … 0.74
…	…
zebra	· · … ·

Every word has a target and a context embedding

Skip-gram learns separate parameter matrices W and C for these two roles.

$\theta =$

W target	dimensions 1 … d			
aardvark	·	·	…	·
apricot	0.31	−0.08	…	0.74
…	…			
zebra	·	·	…	·

Every word has a target and a context embedding

Skip-gram learns separate parameter matrices W and C for these two roles.

$\theta =$

W target	dimensions 1 … d
aardvark	· · … ·
apricot	0.31 −0.08 … 0.74
…	…
zebra	· · … ·

Target lookup in W

$$\mathbf{w} = W[\text{apricot}]$$

apricot used as the target

Every word has a target and a context embedding

Skip-gram learns separate parameter matrices W and C for these two roles.

$\theta =$

W target	dimensions 1 … d
aardvark	· · … ·
apricot	0.31 −0.08 … 0.74
…	…
zebra	· · … ·

C context	dimensions 1 … d
aardvark	· · … ·
jam	−0.12 0.57 … 0.23
…	…
zebra	· · … ·

Target lookup in W

$$\mathbf{w} = W[\text{apricot}]$$

apricot used as the target

Every word has a target and a context embedding

Skip-gram learns separate parameter matrices W and C for these two roles.

$$\theta =$$

W target	**dimensions 1 … d**
aardvark	· · … ·
apricot	0.31 −0.08 … 0.74
…	…
zebra	· · … ·

C context	**dimensions 1 … d**
aardvark	· · … ·
jam	−0.12 0.57 … 0.23
…	…
zebra	· · … ·

Target lookup in W

$$\mathbf{w} = W[\text{apricot}]$$

apricot used as the target

Context lookup in C

$$\mathbf{c} = C[\text{jam}]$$

jam used as context or noise

Every word has a target and a context embedding

Skip-gram learns separate parameter matrices W and C for these two roles.

$\theta =$

W target	dimensions 1 … d			
aardvark	·	·	…	·
apricot	0.31	−0.08	…	0.74
…	…			
zebra	·	·	…	·

C context	dimensions 1 … d			
aardvark	·	·	…	·
jam	−0.12	0.57	…	0.23
…	…			
zebra	·	·	…	·

Target lookup in W

$\mathbf{w} = W[\text{apricot}]$

apricot used as the target

Context lookup in C

$\mathbf{c} = C[\text{jam}]$

jam used as context or noise

Together: $2|V|$ learned vectors, each with d dimensions.

J&M Fig. 5.6 · redrawn

How can *w* and *c* express whether a target–context pair belongs together?

Dot products score target–context compatibility

A larger score supports the hypothesis that the pair occurs together in real text.

$$\mathsf{score}(\mathbf{w}, \mathbf{c}) = \mathbf{c} \cdot \mathbf{w} = \sum_j c_j w_j$$

Multiply corresponding dimensions, then add the contributions.

Illustrative 3-D values

(apricot, jam)

$\mathbf{w} = [0.60,\ 0.20,\ 0.70]$

$\mathbf{c} = [0.50,\ 0.30,\ 0.60]$

$0.30 + 0.06 + 0.42$

$\mathbf{c} \cdot \mathbf{w} = 0.78$

aligned contributions → higher compatibility

Dot products score target–context compatibility

A larger score supports the hypothesis that the pair occurs together in real text.

$$\text{score}(\mathbf{w}, \mathbf{c}) = \mathbf{c} \cdot \mathbf{w} = \sum_j c_j w_j$$

Multiply corresponding dimensions, then add the contributions.

Illustrative 3-D values

(apricot, jam)

$\mathbf{w} = [0.60,\ 0.20,\ 0.70]$

$\mathbf{c} = [0.50,\ 0.30,\ 0.60]$

$0.30 + 0.06 + 0.42$

$\mathbf{c} \cdot \mathbf{w} = 0.78$

aligned contributions → higher compatibility

(apricot, aardvark)

$\mathbf{w} = [0.60,\ 0.20,\ 0.70]$

$\mathbf{c} = [-0.40,\ 0.50,\ -0.30]$

$-0.24 + 0.10 - 0.21$

$\mathbf{c} \cdot \mathbf{w} = -0.35$

opposing contributions → lower compatibility

Dot products score target–context compatibility

A larger score supports the hypothesis that the pair occurs together in real text.

$$\text{score}(\mathbf{w}, \mathbf{c}) = \mathbf{c} \cdot \mathbf{w} = \sum_j c_j w_j$$

Multiply corresponding dimensions, then add the contributions.

Illustrative 3-D values

(apricot, jam)

$\mathbf{w} = [0.60,\ 0.20,\ 0.70]$

$\mathbf{c} = [0.50,\ 0.30,\ 0.60]$

$0.30 + 0.06 + 0.42$

$\mathbf{c} \cdot \mathbf{w} = 0.78$

aligned contributions → higher compatibility

(apricot, aardvark)

$\mathbf{w} = [0.60,\ 0.20,\ 0.70]$

$\mathbf{c} = [-0.40,\ 0.50,\ -0.30]$

$-0.24 + 0.10 - 0.21$

$\mathbf{c} \cdot \mathbf{w} = -0.35$

opposing contributions → lower compatibility

Recall: cosine is a normalized dot product. Skip-gram uses the raw dot product, so both direction and vector magnitude affect the score.

J&M §5.5.1 · Eq. 5.13

Dot products score target–context compatibility

A larger score supports the hypothesis that the pair occurs together in real text.

$$\text{score}(\mathbf{w}, \mathbf{c}) = \mathbf{c} \cdot \mathbf{w} = \sum_j c_j w_j$$

Multiply corresponding dimensions, then add the contributions.

Illustrative 3-D values

(apricot, jam)

$\mathbf{w} = [0.60,\ 0.20,\ 0.70]$

$\mathbf{c} = [0.50,\ 0.30,\ 0.60]$

$0.30 + 0.06 + 0.42$

$\mathbf{c} \cdot \mathbf{w} = 0.78$

aligned contributions → higher compatibility

(apricot, aardvark)

$\mathbf{w} = [0.60,\ 0.20,\ 0.70]$

$\mathbf{c} = [-0.40,\ 0.50,\ -0.30]$

$-0.24 + 0.10 - 0.21$

$\mathbf{c} \cdot \mathbf{w} = -0.35$

opposing contributions → lower compatibility

Recall: cosine is a normalized dot product. Skip-gram uses the raw dot product, so both direction and vector magnitude affect the score.

J&M §5.5.1 · Eq. 5.13

The score ranges from $-\infty$ to ∞. How can it become a probability?

The sigmoid converts compatibility into probability

The dot products from the preceding slide now become probabilities of positive pairs.

Compatibility score

$$s = \mathbf{c} \cdot \mathbf{w}$$

$$\sigma(s) = \frac{1}{1 + \exp(-s)}$$

$$P(+ \mid w, c) = \sigma(\mathbf{c} \cdot \mathbf{w})$$

Carry the same numerical scores through the transformation

apricot with jam

$s = 0.78$

$\sigma(s) = 0.686$

decision midpoint

$s = 0.00$

$\sigma(s) = 0.500$

apricot with matrix

$s = -0.35$

$\sigma(s) = 0.413$

The sigmoid converts compatibility into probability

The dot products from the preceding slide now become probabilities of positive pairs.

Compatibility score

$s = \mathbf{c} \cdot \mathbf{w}$ → $\sigma(s) = \dfrac{1}{1 + \exp(-s)}$ → $P(+ \mid w, c) = \sigma(\mathbf{c} \cdot \mathbf{w})$

Carry the same numerical scores through the transformation

apricot with jam	decision midpoint	apricot with matrix
$s = 0.78$	$s = 0.00$	$s = -0.35$
$\sigma(s) = 0.686$	$\sigma(s) = 0.500$	$\sigma(s) = 0.413$

The sigmoid changes the scale, but preserves the ordering of the scores.

J&M §5.5.1 · Eqs. 5.14–5.16

One target has several context words. How are their probabilities combined?

A context window contributes several probabilities

Conditional independence lets skip-gram multiply the pair probabilities for one target.

Shared target

$w = \text{apricot}$

tablespoon

$p_1 = \sigma(\mathbf{c}_1 \cdot \mathbf{w})$

of

$p_2 = \sigma(\mathbf{c}_2 \cdot \mathbf{w})$

jam

$p_3 = \sigma(\mathbf{c}_3 \cdot \mathbf{w})$

a

$p_4 = \sigma(\mathbf{c}_4 \cdot \mathbf{w})$

A context window contributes several probabilities

Conditional independence lets skip-gram multiply the pair probabilities for one target.

Shared target

$w =$ apricot

tablespoon

$p_1 = \sigma(\mathbf{c}_1 \cdot \mathbf{w})$

of

$p_2 = \sigma(\mathbf{c}_2 \cdot \mathbf{w})$

jam

$p_3 = \sigma(\mathbf{c}_3 \cdot \mathbf{w})$

a

$p_4 = \sigma(\mathbf{c}_4 \cdot \mathbf{w})$

Assumption: after the target is known, the context outcomes are treated as independent.

A context window contributes several probabilities

Conditional independence lets skip-gram multiply the pair probabilities for one target.

Shared target

$w = \text{apricot}$

tablespoon

$p_1 = \sigma(\mathbf{c}_1 \cdot \mathbf{w})$

of

$p_2 = \sigma(\mathbf{c}_2 \cdot \mathbf{w})$

jam

$p_3 = \sigma(\mathbf{c}_3 \cdot \mathbf{w})$

a

$p_4 = \sigma(\mathbf{c}_4 \cdot \mathbf{w})$

Assumption: after the target is known, the context outcomes are treated as independent.

$$P(+ \mid w, c_{1:4}) = \prod_{i=1}^{4} \sigma(\mathbf{c}_i \cdot \mathbf{w}) = p_1 p_2 p_3 p_4$$

J&M Eqs. 5.17–5.18

A context window contributes several probabilities

Conditional independence lets skip-gram multiply the pair probabilities for one target.

Shared target

$w = \text{apricot}$

tablespoon

$p_1 = \sigma(\mathbf{c}_1 \cdot \mathbf{w})$

of

$p_2 = \sigma(\mathbf{c}_2 \cdot \mathbf{w})$

jam

$p_3 = \sigma(\mathbf{c}_3 \cdot \mathbf{w})$

a

$p_4 = \sigma(\mathbf{c}_4 \cdot \mathbf{w})$

Assumption: after the target is known, the context outcomes are treated as independent.

$$P(+ \mid w, c_{1:4}) = \prod_{i=1}^{4} \sigma(\mathbf{c}_i \cdot \mathbf{w}) = p_1 p_2 p_3 p_4$$

J&M Eqs. 5.17–5.18

Logarithms turn the window product into a sum

The same objective becomes additive: every context word contributes one log-probability.

Window probability

$$P = p_1 p_2 p_3 p_4$$

take log

Log window probability

$$\log P = \log p_1 + \log p_2 + \log p_3 + \log p_4$$

The transformation changes the calculation, not the preferred parameters

Because log is increasing, maximizing P also maximizes log P.

J&M Eq. 5.18

How do these context pairs become positive and negative training examples?

A context window supplies positive and negative training pairs

Observed neighbours are positive; sampled vocabulary words provide contrasting negatives.

One ±2 context window around the target

tablespoon | of | **apricot** | jam | a

Observed neighbours → positive pairs

```
(apricot, tablespoon)    +
(apricot, of)    +
(apricot, jam)    +
(apricot, a)    +
```

A context window supplies positive and negative training pairs

Observed neighbours are positive; sampled vocabulary words provide contrasting negatives.

One ±2 context window around the target

tablespoon | of | apricot | jam | a

Observed neighbours → positive pairs

(apricot, tablespoon) +

(apricot, of) +

(apricot, jam) +

(apricot, a) +

Each positive pair receives k noise contrasts

(apricot, jam) +

sample k = 2

(apricot, matrix) −

(apricot, Tolstoy) −

A context window supplies positive and negative training pairs

Observed neighbours are positive; sampled vocabulary words provide contrasting negatives.

One ±2 context window around the target

tablespoon | of | **apricot** | jam | a

Observed neighbours → positive pairs

- `(apricot, tablespoon)` +
- `(apricot, of)` +
- `(apricot, jam)` +
- `(apricot, a)` +

Each positive pair receives k noise contrasts

`(apricot, jam)` +

sample k = 2

- `(apricot, matrix)` −
- `(apricot, Tolstoy)` −

Four observed neighbours give four positive pairs and, with k = 2, eight noise pairs.

J&M §5.5.2

A context window supplies positive and negative training pairs

Observed neighbours are positive; sampled vocabulary words provide contrasting negatives.

One ±2 context window around the target

tablespoon	of	**apricot**	jam	a

Observed neighbours → positive pairs

- `(apricot, tablespoon)` +
- `(apricot, of)` +
- `(apricot, jam)` +
- `(apricot, a)` +

Each positive pair receives k noise contrasts

`(apricot, jam)` +

sample k = 2

- `(apricot, matrix)` −
- `(apricot, Tolstoy)` −

Four observed neighbours give four positive pairs and, with k = 2, eight noise pairs.

J&M §5.5.2

How should SGNS choose the noise words?

The 0.75 power flattens the noise distribution

SGNS transforms word counts and then renormalizes them into sampling probabilities.

Flatten first; normalize second

The 0.75 power flattens the noise distribution

SGNS transforms word counts and then renormalizes them into sampling probabilities.

Flatten first; normalize second

1 Start from counts

$$\text{count}(w)$$

Common words begin with much larger values.

The 0.75 power flattens the noise distribution

SGNS transforms word counts and then renormalizes them into sampling probabilities.

Flatten first; normalize second

1 Start from counts

$$\text{count}(w)$$

Common words begin with much larger values.

2 Apply the 0.75 power

$$\text{count}(w)^{0.75}$$

Large count ratios shrink while ordering is preserved.

The 0.75 power flattens the noise distribution

SGNS transforms word counts and then renormalizes them into sampling probabilities.

Flatten first; normalize second

1 Start from counts

$\text{count}(w)$

Common words begin with much larger values.

2 Apply the 0.75 power

$\text{count}(w)^{0.75}$

Large count ratios shrink while ordering is preserved.

A $100{:}1$ count ratio becomes $100^{0.75}{:}1^{0.75} \approx 31.6{:}1$ before normalization.

Frequent words remain more likely, but rare words receive more sampling mass.

J&M §5.5.2 · Eqs. 5.19–5.20

The 0.75 power flattens the noise distribution

SGNS transforms word counts and then renormalizes them into sampling probabilities.

Flatten first; normalize second

1 Start from counts

$$\text{count}(w)$$

Common words begin with much larger values.

2 Apply the 0.75 power

$$\text{count}(w)^{0.75}$$

Large count ratios shrink while ordering is preserved.

3 Renormalize

$$P_{0.75}(w) = \frac{\text{count}(w)^{0.75}}{\sum_{w'} \text{count}(w')^{0.75}}$$

The transformed values now sum to one.

A $100{:}1$ count ratio becomes $100^{0.75}{:}1^{0.75} \approx 31.6{:}1$ before normalization.

Frequent words remain more likely, but rare words receive more sampling mass.

J&M §5.5.2 · Eqs. 5.19–5.20

The 0.75 power flattens the noise distribution

SGNS transforms word counts and then renormalizes them into sampling probabilities.

Flatten first; normalize second

1 Start from counts

$\text{count}(w)$

Common words begin with much larger values.

2 Apply the 0.75 power

$\text{count}(w)^{0.75}$

Large count ratios shrink while ordering is preserved.

3 Renormalize

$$P_{0.75}(w) = \frac{\text{count}(w)^{0.75}}{\sum_{w'} \text{count}(w')^{0.75}}$$

The transformed values now sum to one.

A $100{:}1$ count ratio becomes $100^{0.75}{:}1^{0.75} \approx 31.6{:}1$ before normalization.

Frequent words remain more likely, but rare words receive more sampling mass.

J&M §5.5.2 · Eqs. 5.19–5.20

Once the positive and negative pairs are assembled, what objective trains W and C?

SGNS maximizes correct-label probability

One case contains one observed context word and k independently sampled noise words.

One sampled training case

target w	positive context c_{pos}	noise context $c_{\text{neg},1}$	noise context $c_{\text{neg},2}$
apricot	jam	matrix	Tolstoy

SGNS maximizes correct-label probability

One case contains one observed context word and k independently sampled noise words.

One sampled training case

target w	positive context c_{pos}	noise context $c_{\text{neg},1}$	noise context $c_{\text{neg},2}$
`apricot`	`jam`	`matrix`	`Tolstoy`

Observed pair should receive +

$$P(+ \mid w, c_{\text{pos}}) = \sigma(\mathbf{c}_{\text{pos}} \cdot \mathbf{w})$$

Each noise pair should receive −

$$P(- \mid w, c_{\text{neg},i}) = \sigma(-\mathbf{c}_{\text{neg},i} \cdot \mathbf{w})$$

SGNS maximizes correct-label probability

One case contains one observed context word and k independently sampled noise words.

One sampled training case

target w	positive context c_{pos}	noise context $c_{\text{neg},1}$	noise context $c_{\text{neg},2}$
apricot	jam	matrix	Tolstoy

Observed pair should receive +

$$P(+ \mid w, c_{\text{pos}}) = \sigma(\mathbf{c}_{\text{pos}} \cdot \mathbf{w})$$

Each noise pair should receive −

$$P(- \mid w, c_{\text{neg},i}) = \sigma(-\mathbf{c}_{\text{neg},i} \cdot \mathbf{w})$$

Correct-label likelihood

$$\mathcal{L} = \sigma(\mathbf{c}_{\text{pos}} \cdot \mathbf{w}) \prod_{i=1}^{k} \sigma(-\mathbf{c}_{\text{neg},i} \cdot \mathbf{w})$$

J&M §5.5.2 · Eq. 5.21

The negative label reverses the compatibility score

For a noise pair, the correct-class probability is the complement of the positive-class probability.

The negative label reverses the compatibility score

For a noise pair, the correct-class probability is the complement of the positive-class probability.

Start with the score for one sampled noise pair

1 Compatibility score

$$s_{\text{neg}} = c_{\text{neg}} \cdot w$$

The negative label reverses the compatibility score

For a noise pair, the correct-class probability is the complement of the positive-class probability.

Start with the score for one sampled noise pair

1 Compatibility score

$$s_{\text{neg}} = c_{\text{neg}} \cdot w$$

2 Positive-class probability

$$P(+ \mid w, c_{\text{neg}}) = \sigma(s_{\text{neg}})$$

The negative label reverses the compatibility score

For a noise pair, the correct-class probability is the complement of the positive-class probability.

Start with the score for one sampled noise pair

1 Compatibility score

$$s_{\text{neg}} = c_{\text{neg}} \cdot w$$

2 Positive-class probability

$$P(+ \mid w, c_{\text{neg}}) = \sigma(s_{\text{neg}})$$

3 A negative label means "not positive"

$$P(- \mid w, c_{\text{neg}}) = 1 - P(+ \mid w, c_{\text{neg}}) = 1 - \sigma(s_{\text{neg}})$$

The negative label reverses the compatibility score

For a noise pair, the correct-class probability is the complement of the positive-class probability.

Start with the score for one sampled noise pair

1 Compatibility score

$$s_{\text{neg}} = c_{\text{neg}} \cdot w$$

2 Positive-class probability

$$P(+ \mid w, c_{\text{neg}}) = \sigma(s_{\text{neg}})$$

3 A negative label means "not positive"

$$P(- \mid w, c_{\text{neg}}) = 1 - P(+ \mid w, c_{\text{neg}}) = 1 - \sigma(s_{\text{neg}})$$

Sigmoid complement identity

$$1 - \sigma(s) = \sigma(-s)$$

The negative label reverses the compatibility score

For a noise pair, the correct-class probability is the complement of the positive-class probability.

Start with the score for one sampled noise pair

1 Compatibility score

$$s_{\text{neg}} = c_{\text{neg}} \cdot w$$

2 Positive-class probability

$$P(+ \mid w, c_{\text{neg}}) = \sigma(s_{\text{neg}})$$

3 A negative label means “not positive”

$$P(- \mid w, c_{\text{neg}}) = 1 - P(+ \mid w, c_{\text{neg}}) = 1 - \sigma(s_{\text{neg}})$$

Sigmoid complement identity

$$1 - \sigma(s) = \sigma(-s)$$

Therefore

$$P(- \mid w, c_{\text{neg}}) = \sigma(-s_{\text{neg}}) = \sigma\big(-(c_{\text{neg}} \cdot w)\big)$$

The negative label reverses the compatibility score

For a noise pair, the correct-class probability is the complement of the positive-class probability.

Start with the score for one sampled noise pair

1 Compatibility score

$$s_{\text{neg}} = c_{\text{neg}} \cdot w$$

2 Positive-class probability

$$P(+ \mid w, c_{\text{neg}}) = \sigma(s_{\text{neg}})$$

3 A negative label means "not positive"

$$P(- \mid w, c_{\text{neg}}) = 1 - P(+ \mid w, c_{\text{neg}}) = 1 - \sigma(s_{\text{neg}})$$

Sigmoid complement identity

$$1 - \sigma(s) = \sigma(-s)$$

Therefore

$$P(- \mid w, c_{\text{neg}}) = \sigma(-s_{\text{neg}}) = \sigma\big(-(c_{\text{neg}} \cdot w)\big)$$

The outer minus negates the scalar score—not a separately stored context vector.

The negative label reverses the compatibility score

For a noise pair, the correct-class probability is the complement of the positive-class probability.

Start with the score for one sampled noise pair

1 Compatibility score

$$s_{\text{neg}} = c_{\text{neg}} \cdot w$$

2 Positive-class probability

$$P(+ \mid w, c_{\text{neg}}) = \sigma(s_{\text{neg}})$$

3 A negative label means “not positive”

$$P(- \mid w, c_{\text{neg}}) = 1 - P(+ \mid w, c_{\text{neg}}) = 1 - \sigma(s_{\text{neg}})$$

Sigmoid complement identity

$$1 - \sigma(s) = \sigma(-s)$$

Therefore

$$P(- \mid w, c_{\text{neg}}) = \sigma(-s_{\text{neg}}) = \sigma\big(-(c_{\text{neg}} \cdot w)\big)$$

The outer minus negates the scalar score—not a separately stored context vector.

Numerical check

Suppose the noise pair already has a low compatibility score:

$$s_{\text{neg}} = -3$$

$$P(+) = \sigma(-3) = 0.047$$

$$P(-) = 1 - 0.047 = 0.953$$

$$P(-) = \sigma(3) = 0.953$$

A negative score gives a high probability to the negative label.

$$P(\text{correct}) = \sigma(ys)$$

y = +1 for observed; y = −1 for noise

The negative label reverses the compatibility score

For a noise pair, the correct-class probability is the complement of the positive-class probability.

Start with the score for one sampled noise pair

1 Compatibility score

$$s_{\text{neg}} = c_{\text{neg}} \cdot w$$

2 Positive-class probability

$$P(+ \mid w, c_{\text{neg}}) = \sigma(s_{\text{neg}})$$

3 A negative label means "not positive"

$$P(- \mid w, c_{\text{neg}}) = 1 - P(+ \mid w, c_{\text{neg}}) = 1 - \sigma(s_{\text{neg}})$$

Sigmoid complement identity

$$1 - \sigma(s) = \sigma(-s)$$

Therefore

$$P(- \mid w, c_{\text{neg}}) = \sigma(-s_{\text{neg}}) = \sigma\big(-(c_{\text{neg}} \cdot w)\big)$$

The outer minus negates the scalar score—not a separately stored context vector.

Numerical check

Suppose the noise pair already has a low compatibility score:

$$s_{\text{neg}} = -3$$

$$P(+) = \sigma(-3) = 0.047$$

$$P(-) = 1 - 0.047 = 0.953$$

$$P(-) = \sigma(3) = 0.953$$

A negative score gives a high probability to the negative label.

$$P(\text{correct}) = \sigma(ys)$$

y = +1 for observed; y = −1 for noise

The negative log gives the SGNS loss

The positive term and all k negative terms become additive contributions.

$$\mathcal{L} = \sigma(\mathbf{c}_{\text{pos}} \cdot \mathbf{w}) \prod_{i=1}^{k} \sigma(-\mathbf{c}_{\text{neg},i} \cdot \mathbf{w})$$

The negative log gives the SGNS loss

The positive term and all k negative terms become additive contributions.

$$\mathcal{L} = \sigma(\mathbf{c}_{\text{pos}} \cdot \mathbf{w}) \prod_{i=1}^{k} \sigma(-\mathbf{c}_{\text{neg},i} \cdot \mathbf{w})$$

take −log

The negative log gives the SGNS loss

The positive term and all k negative terms become additive contributions.

$$\mathcal{L} = \sigma(\mathbf{c}_{\text{pos}} \cdot \mathbf{w}) \prod_{i=1}^{k} \sigma(-\mathbf{c}_{\text{neg},i} \cdot \mathbf{w})$$

take −log

SGNS loss

$$L = -\left[\log \sigma(\mathbf{c}_{\text{pos}} \cdot \mathbf{w}) + \sum_{i=1}^{k} \log \sigma(-\mathbf{c}_{\text{neg},i} \cdot \mathbf{w})\right]$$

The negative log gives the SGNS loss

The positive term and all k negative terms become additive contributions.

$$\mathcal{L} = \sigma(\mathbf{c}_{\text{pos}} \cdot \mathbf{w}) \prod_{i=1}^{k} \sigma(-\mathbf{c}_{\text{neg},i} \cdot \mathbf{w})$$

take −log

SGNS loss

$$L = -\left[\log \sigma(\mathbf{c}_{\text{pos}} \cdot \mathbf{w}) + \sum_{i=1}^{k} \log \sigma(-\mathbf{c}_{\text{neg},i} \cdot \mathbf{w})\right]$$

Positive term

Minimizing L increases $\mathbf{c}_{\text{pos}} \cdot \mathbf{w}$.

Negative terms

Minimizing L decreases every $\mathbf{c}_{\text{neg},i} \cdot \mathbf{w}$.

The negative log gives the SGNS loss

The positive term and all k negative terms become additive contributions.

$$\mathcal{L} = \sigma(\mathbf{c}_{\text{pos}} \cdot \mathbf{w}) \prod_{i=1}^{k} \sigma(-\mathbf{c}_{\text{neg},i} \cdot \mathbf{w})$$

take −log

SGNS loss

$$L = -\left[\log \sigma(\mathbf{c}_{\text{pos}} \cdot \mathbf{w}) + \sum_{i=1}^{k} \log \sigma(-\mathbf{c}_{\text{neg},i} \cdot \mathbf{w})\right]$$

Positive term

Minimizing L increases $\mathbf{c}_{\text{pos}} \cdot \mathbf{w}$.

Negative terms

Minimizing L decreases every $\mathbf{c}_{\text{neg},i} \cdot \mathbf{w}$.

J&M §5.5.2 · Eq. 5.21

The negative log gives the SGNS loss

The positive term and all k negative terms become additive contributions.

$$\mathcal{L} = \sigma(\mathbf{c}_{\text{pos}} \cdot \mathbf{w}) \prod_{i=1}^{k} \sigma(-\mathbf{c}_{\text{neg},i} \cdot \mathbf{w})$$

take −log

SGNS loss

$$L = -\left[\log \sigma(\mathbf{c}_{\text{pos}} \cdot \mathbf{w}) + \sum_{i=1}^{k} \log \sigma(-\mathbf{c}_{\text{neg},i} \cdot \mathbf{w})\right]$$

Positive term

Minimizing L increases $\mathbf{c}_{\text{pos}} \cdot \mathbf{w}$.

Negative terms

Minimizing L decreases every $\mathbf{c}_{\text{neg},i} \cdot \mathbf{w}$.

J&M §5.5.2 · Eq. 5.21

What does one complete forward pass look like with actual numbers?

Running text supplies the positive examples

A context window converts one sentence into labelled target–context pairs.

Sentence

I spread **apricot** jam on warm toast.

window $= 1$ word on each side

observed near apricot

Running text supplies the positive examples

A context window converts one sentence into labelled target–context pairs.

Sentence

`I` `spread` **`apricot`** `jam` `on` `warm` `toast.`

window $= 1$ word on each side

observed near apricot

The target word is paired with every word inside its window

Running text supplies the positive examples

A context window converts one sentence into labelled target–context pairs.

Sentence

`I` `spread` **`apricot`** `jam` `on` `warm` `toast.`

window $= 1$ word on each side

observed near apricot

The target word is paired with every word inside its window

`(apricot, spread)` $y = 1$

observed immediately before

Running text supplies the positive examples

A context window converts one sentence into labelled target–context pairs.

Sentence

`I` `spread` **`apricot`** `jam` `on` `warm` `toast.`

window $= 1$ word on each side

observed near apricot

The target word is paired with every word inside its window

(apricot, spread)	$y = 1$	observed immediately before
(apricot, jam)	$y = 1$	observed immediately after

Running text supplies the positive examples

A context window converts one sentence into labelled target–context pairs.

Sentence

`I` `spread` **`apricot`** `jam` `on` `warm` `toast.`

window $= 1$ word on each side

observed near apricot

The target word is paired with every word inside its window

(apricot, spread)	$y = 1$	observed immediately before
(apricot, jam)	$y = 1$	observed immediately after

These labels come from the text itself: no human annotator is required.

J&M §5.5.2

Negative sampling creates the contrasting examples

Keep one observed pair, then sample words that did not occupy that context position.

One small training batch		*meaning of the label*
`(apricot, jam)`	$y = 1$	jam occurred beside apricot
`(apricot, aardvark)`	$y = 0$	sampled noise word
`(apricot, tractor)`	$y = 0$	sampled noise word
`(apricot, ocean)`	$y = 0$	sampled noise word

Negative sampling creates the contrasting examples

Keep one observed pair, then sample words that did not occupy that context position.

One small training batch — *meaning of the label*

Pair	Label	Meaning of the label
`(apricot, jam)`	$y = 1$	jam occurred beside apricot
`(apricot, aardvark)`	$y = 0$	sampled noise word
`(apricot, tractor)`	$y = 0$	sampled noise word
`(apricot, ocean)`	$y = 0$	sampled noise word

“Negative” means not observed in this context window. It does not mean the words can never be related.

J&M §5.5.2

Negative sampling creates the contrasting examples

Keep one observed pair, then sample words that did not occupy that context position.

One small training batch

		meaning of the label
`(apricot, jam)`	$y = 1$	jam occurred beside apricot
`(apricot, aardvark)`	$y = 0$	sampled noise word
`(apricot, tractor)`	$y = 0$	sampled noise word
`(apricot, ocean)`	$y = 0$	sampled noise word

“Negative” means not observed in this context window. It does not mean the words can never be related.

J&M §5.5.2

Each labelled pair selects two embedding rows

The target comes from W; every candidate context word comes from C.

Target lookup

$W[\text{apricot}]$

$$\mathbf{w} = [0.60,\ 0.20,\ 0.70]$$

the same target row is reused

Context lookups

$C[\text{jam}]$ $[0.50,\ 0.30,\ 0.60]$

Each labelled pair selects two embedding rows

The target comes from W; every candidate context word comes from C.

Target lookup

$W[\text{apricot}]$

$$\mathbf{w} = [0.60,\ 0.20,\ 0.70]$$

the same target row is reused

Context lookups

$C[\text{jam}]$ $[0.50,\ 0.30,\ 0.60]$

$C[\text{aardvark}]$ $[-0.40,\ 0.50,\ -0.30]$

Each labelled pair selects two embedding rows

The target comes from W; every candidate context word comes from C.

Target lookup

$W[\text{apricot}]$

$$\mathbf{w} = [0.60,\ 0.20,\ 0.70]$$

the same target row is reused

Context lookups

$C[\text{jam}]$	$[0.50,\ 0.30,\ 0.60]$
$C[\text{aardvark}]$	$[-0.40,\ 0.50,\ -0.30]$
$C[\text{tractor}]$	$[-0.50,\ -0.20,\ 0.10]$

Each labelled pair selects two embedding rows

The target comes from W; every candidate context word comes from C.

Target lookup

$W[\text{apricot}]$

$\mathbf{w} = [0.60,\ 0.20,\ 0.70]$

the same target row is reused

Context lookups

$C[\text{jam}]$	$[0.50,\ 0.30,\ 0.60]$
$C[\text{aardvark}]$	$[-0.40,\ 0.50,\ -0.30]$
$C[\text{tractor}]$	$[-0.50,\ -0.20,\ 0.10]$
$C[\text{ocean}]$	$[0.10,\ -0.50,\ -0.40]$

Each labelled pair selects two embedding rows

The target comes from W; every candidate context word comes from C.

Target lookup

$W[\text{apricot}]$

$$\mathbf{w} = [0.60,\ 0.20,\ 0.70]$$

the same target row is reused

Context lookups

$C[\text{jam}]$	$[0.50,\ 0.30,\ 0.60]$
$C[\text{aardvark}]$	$[-0.40,\ 0.50,\ -0.30]$
$C[\text{tractor}]$	$[-0.50,\ -0.20,\ 0.10]$
$C[\text{ocean}]$	$[0.10,\ -0.50,\ -0.40]$

Four pairs → one W row and four C rows. The word apricot is not re-embedded four times.

J&M §5.5.2

Each labelled pair selects two embedding rows

The target comes from W; every candidate context word comes from C.

Target lookup

$W[\text{apricot}]$

$\mathbf{w} = [0.60,\ 0.20,\ 0.70]$

the same target row is reused

Context lookups

$C[\text{jam}]$	$[0.50,\ 0.30,\ 0.60]$
$C[\text{aardvark}]$	$[-0.40,\ 0.50,\ -0.30]$
$C[\text{tractor}]$	$[-0.50,\ -0.20,\ 0.10]$
$C[\text{ocean}]$	$[0.10,\ -0.50,\ -0.40]$

Four pairs → one W row and four C rows. The word apricot is not re-embedded four times.

J&M §5.5.2

$W[\text{apricot}]$

$\mathbf{w} = [0.60,\ 0.20,\ 0.70]$

the same target row is reused

$C[\text{jam}]$ $[0.50,\ 0.30,\ 0.60]$

$C[\text{aardvark}]$ $[-0.40,\ 0.50,\ -0.30]$

$C[\text{tractor}]$ $[-0.50,\ -0.20,\ 0.10]$

$C[\text{ocean}]$ $[0.10,\ -0.50,\ -0.40]$

Dot products score the four candidate pairs

For each pair, multiply matching dimensions and add the three contributions.

$$\text{score}(\mathbf{w}, \mathbf{c}) = \mathbf{c} \cdot \mathbf{w}$$

`jam`	$0.50(0.60) + 0.30(0.20) + 0.60(0.70) = 0.30 + 0.06 + 0.42$	$\text{score} = 0.78$
`aardvark`	$-0.40(0.60) + 0.50(0.20) - 0.30(0.70) = -0.24 + 0.10 - 0.21$	$\text{score} = -0.35$
`tractor`	$-0.50(0.60) - 0.20(0.20) + 0.10(0.70) = -0.30 - 0.04 + 0.07$	$\text{score} = -0.27$
`ocean`	$0.10(0.60) - 0.50(0.20) - 0.40(0.70) = 0.06 - 0.10 - 0.28$	$\text{score} = -0.32$

$W[\text{apricot}]$

$\mathbf{w} = [0.60,\ 0.20,\ 0.70]$

the same target row is reused

$C[\text{jam}]$	$[0.50,\ 0.30,\ 0.60]$
$C[\text{aardvark}]$	$[-0.40,\ 0.50,\ -0.30]$
$C[\text{tractor}]$	$[-0.50,\ -0.20,\ 0.10]$
$C[\text{ocean}]$	$[0.10,\ -0.50,\ -0.40]$

J&M §5.5.2

The sigmoid turns each score into a pair probability

Apply the same nonlinear transformation independently to all four scores.

$$p(y=1 \mid w, c) = \sigma(\mathbf{c} \cdot \mathbf{w}) \quad \text{where} \quad \sigma(s) = \frac{1}{1+\exp(-s)}$$

pair	score s	σ(s)	label y	reading
(apricot, jam)	0.78	0.686	1	*higher is better*
(apricot, aardvark)	−0.35	0.413	0	*lower is better*
(apricot, tractor)	−0.27	0.433	0	*lower is better*
(apricot, ocean)	−0.32	0.421	0	*lower is better*

The sigmoid turns each score into a pair probability

Apply the same nonlinear transformation independently to all four scores.

$$p(y = 1 \mid w, c) = \sigma(\mathbf{c} \cdot \mathbf{w}) \quad \text{where} \quad \sigma(s) = \frac{1}{1 + \exp(-s)}$$

pair	score s	σ(s)	label y	reading
(apricot, jam)	0.78	0.686	1	*higher is better*
(apricot, aardvark)	−0.35	0.413	0	*lower is better*
(apricot, tractor)	−0.27	0.433	0	*lower is better*
(apricot, ocean)	−0.32	0.421	0	*lower is better*

The sigmoid changes the scale but preserves the ordering of the scores.

J&M §5.5.2

The forward pass ends with one combined SGNS loss

Reward the observed pair for high probability and every noise pair for low probability.

Observed pair

$$-\log \sigma(\mathbf{c}_{\text{jam}} \cdot \mathbf{w})$$

$$= -\log(0.685680)$$

$$= 0.377344$$

small when the positive probability is high

The forward pass ends with one combined SGNS loss

Reward the observed pair for high probability and every noise pair for low probability.

Observed pair

$$-\log\sigma(\mathbf{c}_{\text{jam}}\cdot\mathbf{w})$$

$$=-\log(0.685680)$$

$$=0.377344$$

small when the positive probability is high

Three noise pairs

`aardvark` $-\log(1-\sigma(-0.35))=0.533382$

`tractor` $-\log(1-\sigma(-0.27))=0.567232$

`ocean` $-\log(1-\sigma(-0.32))=0.545893$

noise subtotal = 1.646507

The forward pass ends with one combined SGNS loss

Reward the observed pair for high probability and every noise pair for low probability.

Observed pair

$$-\log \sigma(\mathbf{c}_{\text{jam}} \cdot \mathbf{w})$$

$$= -\log(0.685680)$$

$$= 0.377344$$

small when the positive probability is high

Three noise pairs

`aardvark` $-\log(1-\sigma(-0.35)) = 0.533382$

`tractor` $-\log(1-\sigma(-0.27)) = 0.567232$

`ocean` $-\log(1-\sigma(-0.32)) = 0.545893$

noise subtotal = 1.646507

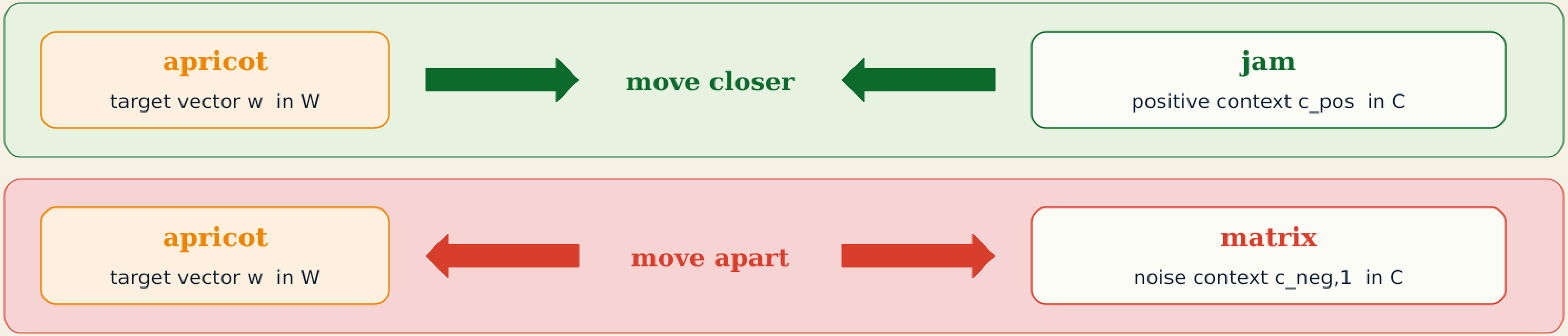
L = 0.377344 + 1.646507 = 2.023851 ≈ 2.024

Forward pass complete: no parameter has been changed yet.

J&M §5.5.2 · Eq. 5.20

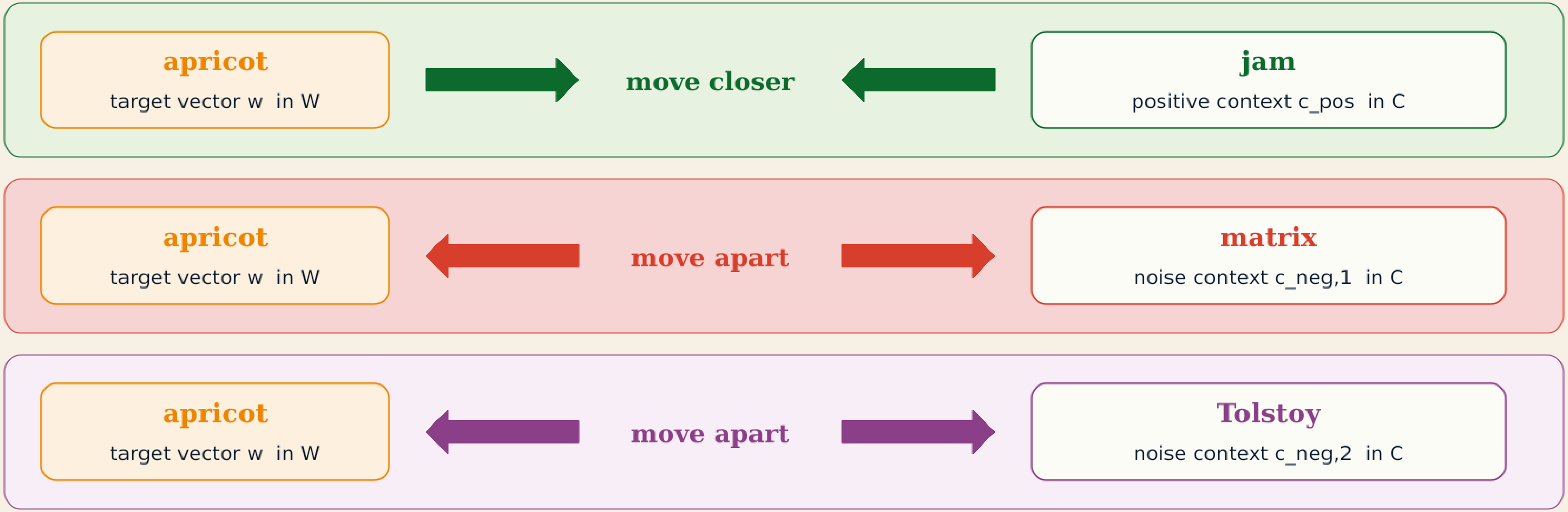
SGD pulls neighbours closer and repels noise

Figure 5.7 visualizes how one update changes the sampled target and context vectors.



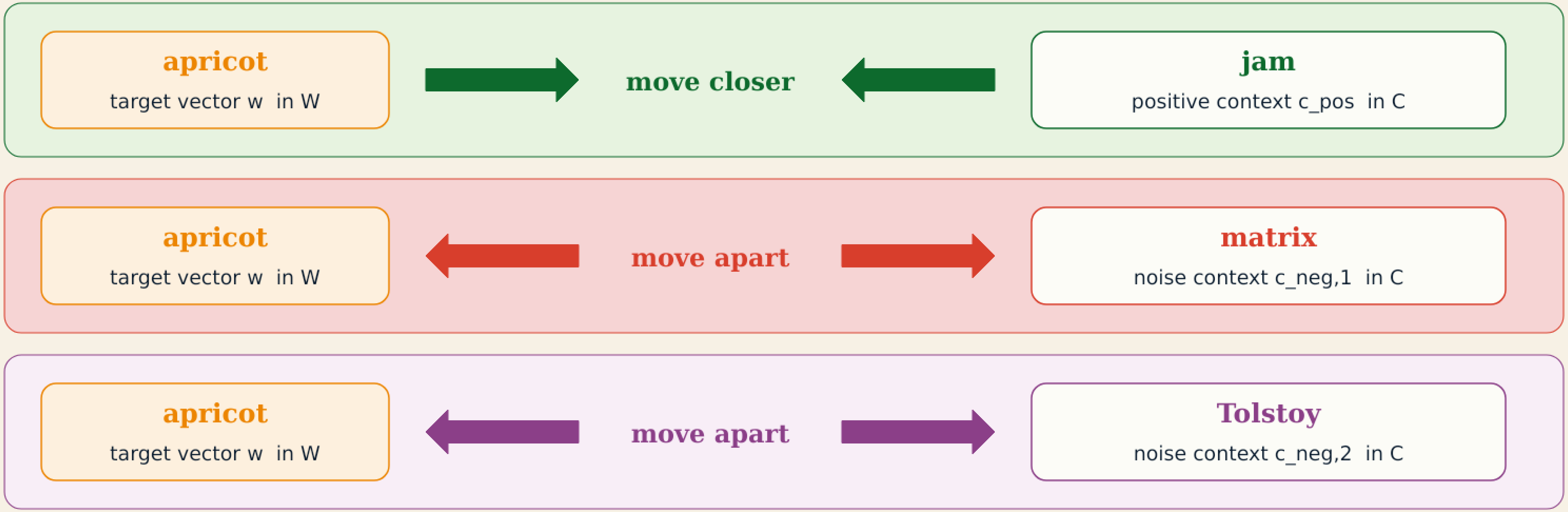
SGD pulls neighbours closer and repels noise

Figure 5.7 visualizes how one update changes the sampled target and context vectors.



SGD pulls neighbours closer and repels noise

Figure 5.7 visualizes how one update changes the sampled target and context vectors.

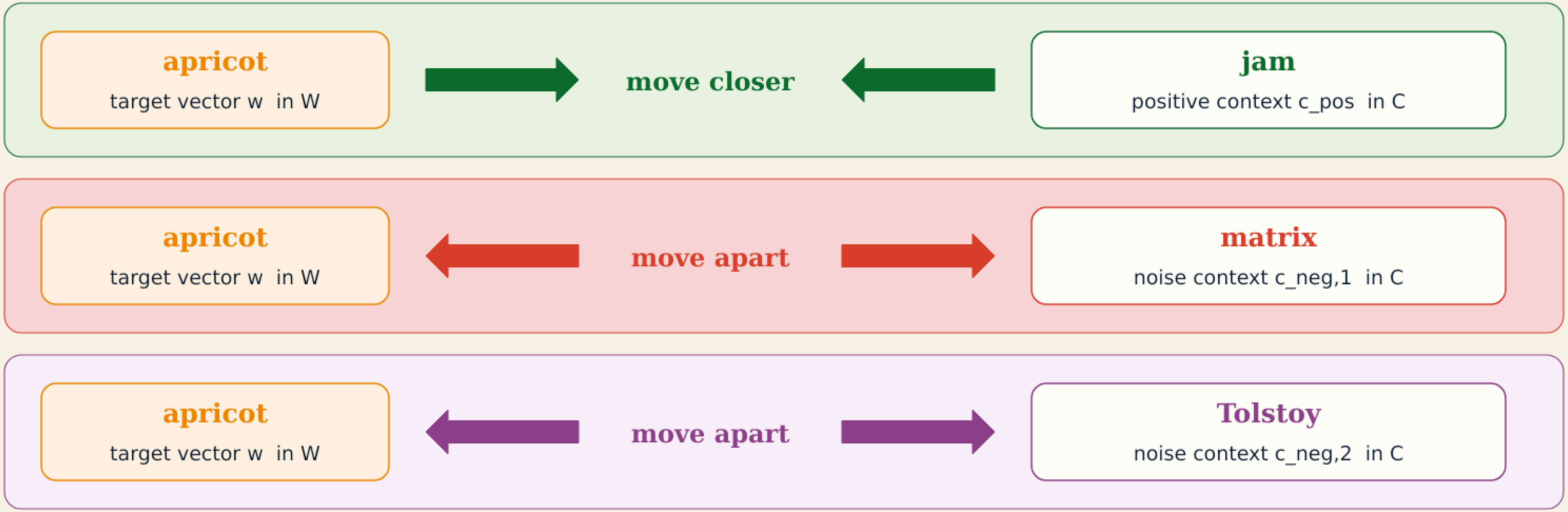


One target update combines the positive pull and both negative pushes.

J&M Fig. 5.7 · redrawn

SGD pulls neighbours closer and repels noise

Figure 5.7 visualizes how one update changes the sampled target and context vectors.



One target update combines the positive pull and both negative pushes.

J&M Fig. 5.7 · redrawn

The geometry is intuitive. What scalar coefficients determine each movement size?